The Halloween Lock: Reverse Engineering

Creator: Het Patel Points: 400

Description

A single stage, static analysis only crackme. Input is not accepted. The password check compares against a target that was never derived from a real string, so guessing and brute force are both dead ends by design. The real flag is a separate 30-byte array pre-encoded in the binary; recovering it means disassembling a four-stage transform (XOR against a computed key, a position-based rotate, a position-based add, a running XOR chain) and running it backwards in a script. A plaintext decoy flag in strings output catches anyone who skips disassembly.

Prompt

A door in the old manor is sealed by a lock that answers to no key — and, it turns out, to no spoken word either, no matter how many you try. It was never built to open that way. But it carries something folded up inside itself, and locks can be taken apart.

Flag format: flame{...}

Hints

1. The real flag isn't unlocked by the prompt at all. It's already sitting in the binary, pre-scrambled. Your job is to find it and unscramble it.

2. Disassemble the binary. There's a block of code that only runs if your passphrase matches but it mathematically can't. Read that code anyway. It's compiled in and fully inspectable even though the program itself can never reach it, and it shows you exactly how to undo what was done to the real flag.
3. The scrambling isn't a single operation. It's several applied in a specific order, and each one depends on the byte's position in the array. Disassemble closely enough to find all of them and the order they run in. Decoding is the same steps, same key, just unwound in reverse

Solution

1. `strings haunted_lock` shows `flame{hehe_its_not_that_simple}`. It's the decoy.
2. Try entering any passphrase at the prompt. The program always refuses, for any input.
3. Disassemble: `objdump -d -Mintel halloween_lock`. The binary is small and stripped, with every function inlined into one block, so there are no function names. Work by instruction pattern instead.
4. Find the key routine (0x4011c6): mov eax,0x37, then a loop that does xor edx,0xffffffa5 / store byte / add eax,0x1d, 16 times. That's key[j] = (0x37 + 0x1D*j) ^ 0xA5 for j = 0..15.
5. Find the transform applied to your input (~0x401350–0x4013a0): for each byte i it does xor with key[i%16], rol by (i%5)+1, add with (i*7+13)&0xFF, then xor with the previous output byte (a running chain, starting from IV 0x6E). This matches the four stages named in the hint.
6. That result is fed to memcmp at 0x4013b9 against a 30-byte target at 0x4021d0 — confirm in the bytes that this target isn't the real transform of anything typable; it's filler. This whole prompt-and-check path is a dead end by design.
7. Right after the (unreachable) success branch, a second loop runs starting at 0x4013fd (IV load, mov esi,0x6e): same key,

same rotate amounts, but it subtracts instead of adds and XORs against the previous input byte instead of chaining forward the inverse of the first transform. It decodes a 30-byte array sitting at 0x4021f0 in .rodata. That array is the real sealed flag, pre-encoded offline and never present anywhere as plaintext.

8. Recreate the inverse transform in a script:

```
KEY = bytes(((0x37 + 0x1D * j) ^ 0xA5) & 0xFF for j in range(16))
IV = 0x6E

sealed = [152,18,218,92,202,150,149,73,100,109,131,159,51,73,102,10,
          114,186,191,106,249,145,187,176,193,114,155,84,161,68]

def ror8(x, r):
    r &= 7
    return ((x >> r) | (x << (8 - r))) & 0xFF

prev = IV
out = bytearray()
for i, b in enumerate(sealed):
    s3 = b ^ prev
    s2 = (s3 - ((i * 7 + 13) & 0xFF)) & 0xFF
    r = (i % 5) + 1
    s1 = ror8(s2, r)
    out.append(s1 ^ KEY[i % 16])
    prev = b
```

Flag

flame{st4t1c_4n4lys1s_0r_bust} <- correct answer

flame{hehe_its_not_that_simple} <- decoy flag